YOUR GUIDE · READ THIS BEFORE THE APPLICATION AND ANY INTERVIEW

How our two AI systems work, step by step

The context harness (Obsidian ontology graph) and the video pipeline (yt-dlp, FFmpeg, Whisper, Qwen, Claude): what each part does, which file does it, which models run where, the tricks that keep it working, and how to explain it.

> **Your two application lines (125 characters each)**
> 1. Built an Obsidian ontology-graph harness: shared multi-account context, token use cut from 100% to 37%; aiding 20+ delegates.
> 2. Built an AI video pipeline for YouTube, Instagram and Telegram; cut time to grasp a 15-minute video to about 300-400 seconds.

The public code and the university README are in the GitHub repository. This guide stays private.

Part 0. The one-minute version

Problem 1: an AI assistant like Claude forgets everything when a chat ends. **Our fix:** its memory lives in Obsidian notes that link to each other like a map (an "ontology graph"). Every new chat reads the same five short notes, then starts work. The notes are on GitHub, so every PC and every account gets the same memory. Every mistake is written down and turned into a rule, so the system gets better by itself.

Problem 2: Claude cannot watch videos, and showing it every frame costs a fortune. **Our fix:** free programs on our own PC do the watching and listening (download, transcribe, find cuts, describe frames). They write one short summary, and Claude reads only that. An 18-minute film costs about 18,000 tokens instead of 255,000 to 791,000.

Part 1. The context harness

1.1 What "ontology graph" means here

An ontology is a map of things and how they relate. In our vault every note is a **node**. Two kinds of **edge** connect nodes:

- **Parent edge:** the top of each note says `up: "[[INDEX]]"`, meaning "I belong under INDEX". Together these make a tree: INDEX → projects → each project's STATE, run-log and notes.
- **Cross edge:** any `[[Note name]]` in the text. Example: a protocol links to the lesson it came from.

Open Obsidian's Graph view and you can see it: each project has its own colour, and the Harness notes sit in the middle because everything links to them.

One vault (the Fable knowledge vault) is **built by code**. `ontology.py` lists domains (e.g. "Colour") and concepts inside them (e.g. "Colour Combinations"), each with a keyword pattern and a definition. `build_vault.py` reads thousands of scraped YouTube/Reddit questions, matches each to concepts using the keywords, and writes one note per concept with its questions, links and an overview canvas. To change the map you edit `ontology.py` and rebuild. You never edit the notes by hand.

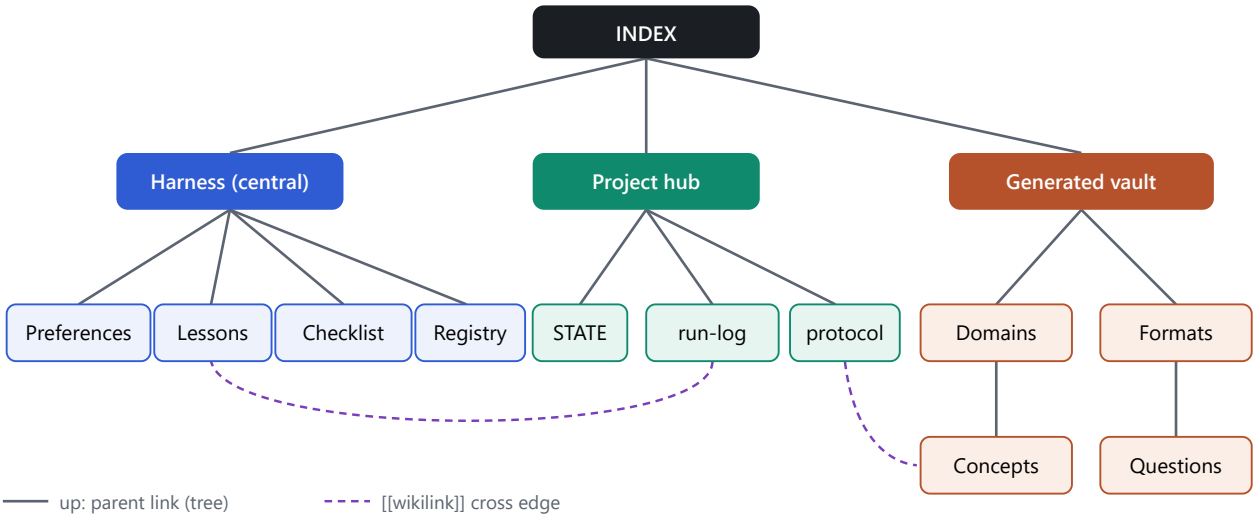


Figure 1. The vault as an ontology: parent links make a tree from one index; cross links connect a project's protocol to the lessons it came from, or a concept to the questions about it.

1.2 The harness files (what each one is for)

File	Job
`Harness/Preferences.md`	How the user wants work done (e.g. "local models do the bulk, Claude does the thinking"). Every rule has the user's words and the date.
`Harness/Vault Registry.md`	Every vault, where it lives, and which file to open first.
`Harness/Lessons.md`	Every mistake as one row: symptom, cause, fix.
`Harness/Eval Checklist.md`	What must be true before saying "done" (output checked, PDF pages looked at, secrets not committed…).
`_Harness/Resume Protocol.md`	Per vault: "read this first after a cleared chat": what is running, how to check it, what is next.
`Projects/<name>/STATE.md`	Where the project is right now. Short, rewritten after every step.
`Projects/<name>/run-log.md`	Every step ever taken, newest at the end. Append only.
`Resources/model-log.md`	Which AI model did which task and how well (the user's own rating).
`CLAUDE.md` (global rules)	Loaded automatically by every Claude Code session. It says: never rebuild context from the chat; read the notes in this order.

1.3 Context management, step by step

1 New chat starts. Claude runs `hostname` to know which PC it is on, and labels the chat *topic - account - device*.

2 It pulls the latest notes: `git pull --rebase --autostash` in the vault.

3 It reads Preferences → Vault Registry → the vault's Resume Protocol → the project's STATE → the last 15 lines of the run-log. Then it stops reading.

4 It works. Heavy data goes through local tools first (Part 2); Claude reads only their short outputs.

5 After each chunk it writes the run-log and STATE, adds any lesson, runs the Eval Checklist, and commits and pushes only the changed notes after scanning them for secrets.

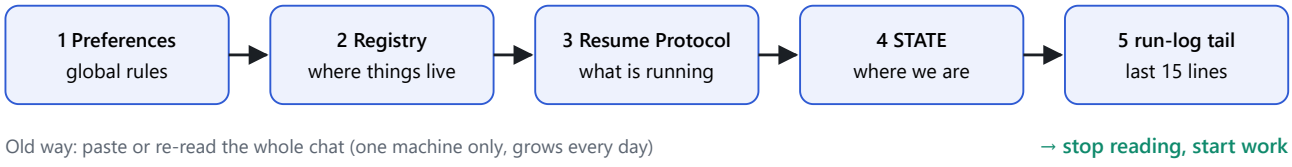


Figure 2. The resume path. Each note is short and current because the work loop rewrites STATE after every step.

Why it saves tokens: the five notes stay small, while the project keeps growing. Measured on our vaults: a new chat loads 3k tokens of the MUN Academy vault's 719k (0.4%), 5.5k of Web Scraping's 126k (4%), and 7.7k of

Clipping's 53k (15%). Our overall estimate for daily work is about 37% of the old way. Say "estimate" when you quote 37%: it is our number, not an outside study.

1.4 Relearning, step by step

1. Something goes wrong, or the user corrects Claude.
2. One row goes into Lessons: what happened, why, the fix.
3. If the same lesson happens a **second** time, it is promoted: a script that makes the mistake impossible, a line in the Eval Checklist, or a rule in Preferences.
4. The Eval Checklist runs before every "done", so the rule is enforced from then on.

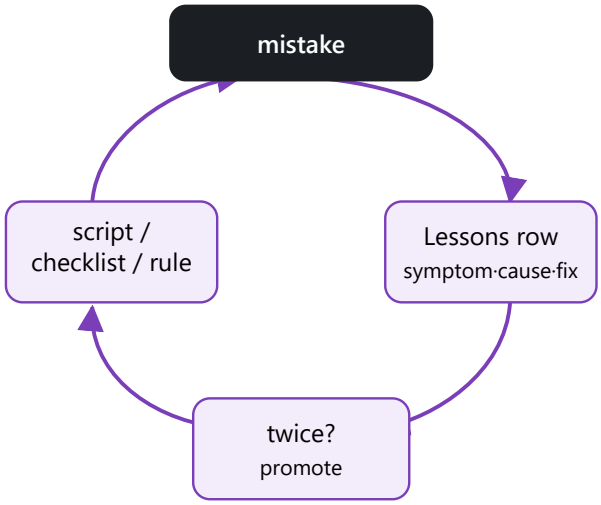


Figure 3. The relearning loop.

Real example to tell: our local vision model labelled 39% of a film "screen capture", but on 50 shots we checked by hand it was right only 40% of the time. Cause: we had tested it on 6 similar frames, and the labels had no definitions. Fix: a test set covering every label, and one-line definitions in the prompt. Accuracy went to 76%, and "test any judge on a gold set first" became a permanent rule.

Protocol first, then delegate: a new kind of job runs first with the strongest model (Opus) and the user. During those rounds the exact protocol is written: every measurement, its script, thresholds, and what an agent may read. After that, cheaper models (Sonnet, Haiku) run it from the protocol. Example: `analysis-protocol.md` for films.

1.5 Many accounts and PCs, one memory

Every vault is a **private GitHub repository**. The global rules file is in one too. Any PC that clones them, with any Claude account signed in, gets the same notes and the same rules. That is what "shared multi-account context" means. Safety rules: commit only changed notes, never the editor's per-device settings, scan for keys and cookies first, and on a conflict stop and report (never force-push). The script is `context-harness/scripts/sync-vaults.sh`.

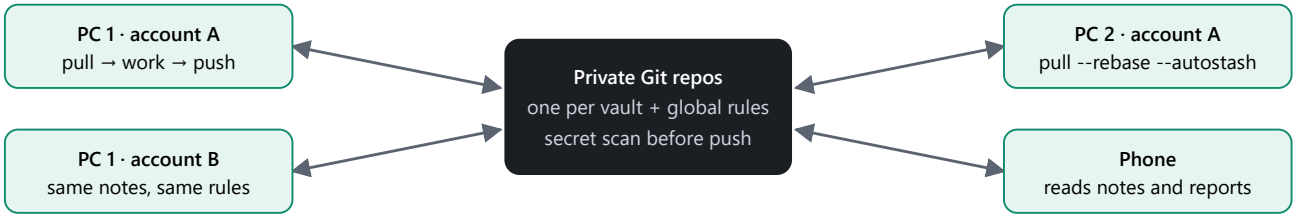


Figure 4. Every machine and assistant account starts from the same notes. Only changed notes are committed (never per-device editor files), each commit carries the device name, and a conflict stops the sync instead of overwriting work.

Part 2. The video pipeline

2.1 The big idea

A video = **sound + pictures on one clock**. We split them, measure each with free local programs, and join the results by time. Claude never watches the video. It reads a one-page summary, the transcript and a few contact sheets.

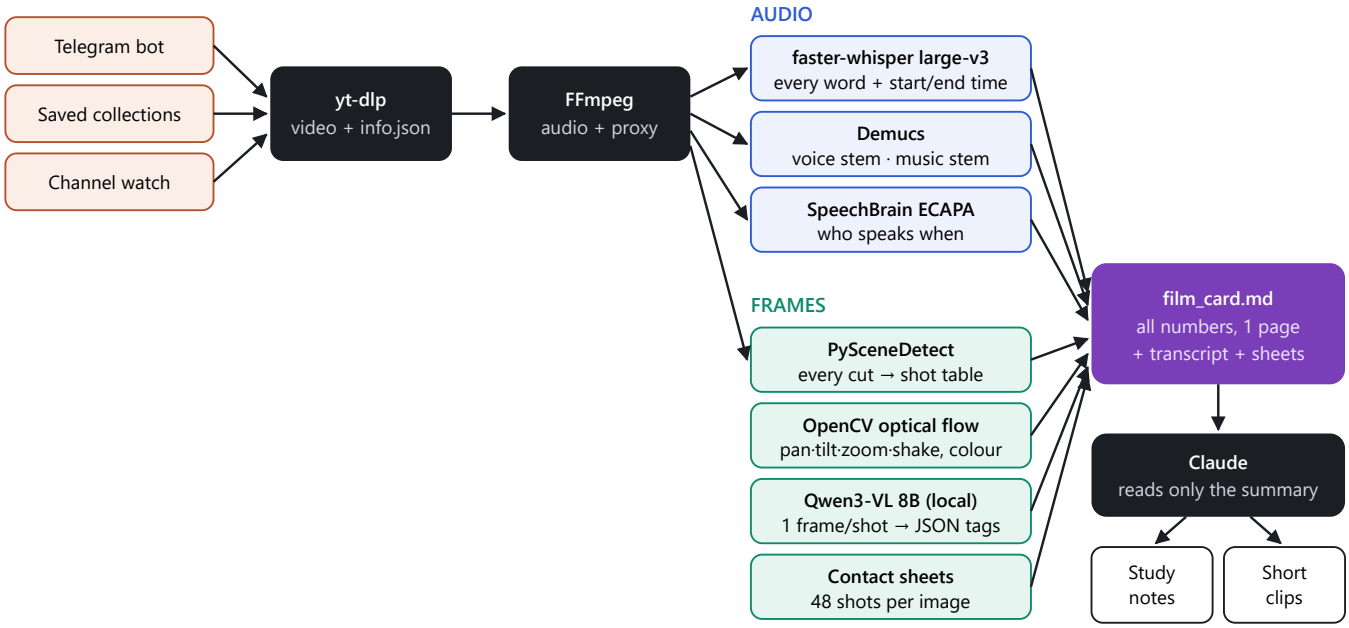


Figure 5. Pipeline architecture. Everything left of the summary card runs locally on the PC; only the card, transcript and sheets reach Claude.

2.2 Step by step, with the file that does it

1 **Send the link.** You send an Instagram, YouTube or TikTok link (or a video under 20 MB) to our private Telegram bot. `1-intake/reel_bot.py` checks the link strictly (so a fake domain like `instagram.com.evil.com` is rejected), strips tracking codes, and only obeys the paired owner (pairing uses a one-time 6-digit code shown on the PC). Other ways in: `collect_links.py` reads our saved Instagram collections; `loop.py` runs every few minutes: collect → download → analyse → delete old videos.

2 **Download.** `dl.py` runs **yt-dlp** (section 2.3) and saves the video, an `info.json` (title, views, likes, chapters, "most replayed" heatmap) and subtitles.

3 **Split.** `run_film.ps1` uses **FFmpeg** to make three files: 16 kHz mono audio (what Whisper wants), 44 kHz stereo audio (for music analysis), and a small 960-pixel proxy video (fast to scan).

4 **Find every cut.** `shots.py` runs **PySceneDetect** to find every cut. It also runs **OpenCV optical flow** on every frame to measure camera moves (pan, tilt, push-in, shake, parallax), and saves the brightness, saturation, a 5-colour palette and one middle frame per shot.

5 **Contact sheets.** `sheets.py` puts 48 shot frames on one image, each with its time and camera move, so a person or Claude can "see" 48 shots in one look.

6 **Words.** `transcribe.py` runs **faster-whisper large-v3** on the GPU, which writes every word with its start and end time. If the GPU fails, it falls back to the CPU.

7 **Voice vs music. Demucs** splits the audio into a voice track and a music track. `audio.py` then measures: words per minute, pauses (and the sentence before each long pause, which is usually a hook line), voice pitch, how far the music is ducked under the voice, music change points, and whether cuts land on sound hits more often than chance.

8 **Who speaks.** `voices.py` turns each sentence into a "voice fingerprint" with **SpeechBrain ECAPA** and groups similar ones. It even spots a phone-line voice (no bass, no treble).

9 **Describe frames.** `tag.py` sends one frame per shot to **Qwen3-VL 8B** running locally in **Ollama**. It answers in fixed JSON: medium (3D render, archival video, document, map…), shot size, angle, subject, lighting, on-screen text, effects, and how it was probably made. Temperature 0 keeps answers consistent. The script is resume-safe: it skips shots already done.

10 **One-page summary.** `film_card.py` puts every number into `film_card.md`, chapter by chapter. Claude reads the card, the transcript and the sheets, then writes the study notes. Real example: `example-output/film_card.md`.

2.3 How yt-dlp works (and the flags we use)

yt-dlp is a free, open-source downloader with an "extractor" for each site. For a link it: (1) loads the page or the site's internal API, (2) finds the list of available streams (YouTube hides the addresses behind a JavaScript puzzle, which is why we installed **Node.js** and pass `--js-runtimes node`), (3) picks the format you asked for, (4) downloads video and audio separately and (5) merges them with FFmpeg.

Flag	What it does for us
`-f "bv*[height<=1080]+ba/b"`	Best video up to 1080p + best audio; if that is not possible, the best single file
`--merge-output-format mkv`	Join video and audio into one file
`--write-info-json`	Save title, views, likes, chapters, heatmap, tags
`--write-subs --write-auto-subs --sub-langs "en.*"`	Save English subtitles if they exist
`--write-comments` + `--extractor-args "youtube:comment_sort=top;max_comments=60"`	Top 60 comments (we keep only the text and likes, no usernames)
`--cookies cookies.txt`	Use our own logged-in session (needed for Instagram)
`--match-filter "duration<=150"`	Skip anything longer than 150 s in the Shorts study
`--sleep-interval 0.5 --max-sleep-interval 1`	A small random pause between items
`--print after_move:...` + `--no-simulate`	Print the final path. Careful: `--print` alone means "pretend", so nothing downloads without `--no-simulate`

2.4 How we avoid getting blocked

Sites block clients that look like bots. We avoid that by looking like one careful human, not by hiding:

- **Real session:** cookies exported from our own logged-in Chrome with the open-source "Get cookies.txt LOCALLY" extension. No password is ever typed into a script. The cookie file is secret: never printed, never committed.
- **Home internet:** Instagram distrusts datacentre IPs (servers) far more than home IPs, so it all runs on our PC, not a cloud server.
- **Slow:** at least 30 s between Instagram downloads; one download and one analysis at a time.
- **Back off:** HTTP 429 ("too many requests") = pause Instagram for 10 minutes. HTTP 400 from Instagram's video server is treated the same way.
- **Stop on a block:** a login challenge, "sign in to confirm you're not a bot" or a 403 writes a `STOP` file, and the run ends until a human looks. One age-restricted video is skipped without stopping, because it is not a block.
- **No proxies, no CAPTCHA solvers, no fake fingerprints** in this code. Keep it that way in anything you show a university.

2.5 Local models and how one small GPU runs them

Model	Where	Used for
faster-whisper large-v3	GPU (CPU fallback)	Speech to text with word times; `small` or `large-v3-turbo` for bulk work on CPU
Demucs htdemucs	GPU	Voice / music separation
SpeechBrain ECAPA	CPU	Speaker fingerprints
Qwen3-VL 8B instruct	GPU, via Ollama	Describing frames as JSON
Qwen3 4B	GPU, via Ollama	Backup judge for clip moments, voice check of written copy
Claude Haiku / Sonnet	Cloud, subscription	Haiku scores 25 moments in one call; Sonnet writes hooks and titles; capped at 20 calls a night

The GPU trick: our GPU has 6–8 GB, which fits one big model at a time. So `run_film.ps1` has two stages: `-Stage cpu` (download, split, cuts, sheets) and `-Stage gpu` (Whisper, then Demucs, then the vision model). Before Whisper it runs `ollama stop` to free the GPU. Every heavy step first checks free RAM and VRAM and waits if they are low.

2.6 Long video → short clips (the clipper)

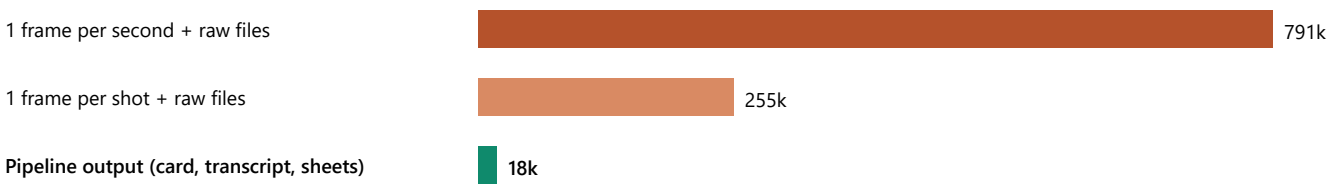
1. `moments/finder.py` tries every 25–62 s window that starts on a new sentence (not "and…", "but…") and ends on a finished one. It scores loudness bursts, reaction words ("no way", "let's go"), a hook at the start, a payoff at the end, speech density and the YouTube heatmap, and subtracts points for silence and filler words. Picks must be 3+ minutes apart. A quick frame check pushes down boring screens (menus, lobbies).
2. Claude Haiku scores the top 25 in **one** call; if the budget is spent, local Qwen3 4B judges instead.
3. `cutter/cutter.py`: FFmpeg makes a 1080×1920 clip (facecam on top, game below, or a blurred background), burns in captions where the spoken word turns yellow, and evens out the loudness.
4. `writer/writer.py`: Claude Sonnet writes a label, 3 hooks, a title and a caption from our study of 1,000+ clips. A Voice Check blocks long text, banned topics (e.g. accusing someone of cheating) and quotes that are not really in the clip.
5. A human approves every clip in a review note. Nothing is posted automatically.

2.7 Problems we hit and how we got around them

Problem	Fix
Python downloads failed with certificate errors	The antivirus re-signs HTTPS traffic. We use the `truststore` package so Python trusts the Windows certificate store. Never turn certificate checks off.
`yt-dlp.exe` blocked by Windows Application Control	Run it as a Python module: `python -m yt_dlp`
Whisper crashed on PyAV 19 (a removed argument)	Read the WAV file ourselves with numpy, or patch the call inside the run
cuBLAS/cuDNN DLLs not found for the GPU	Add the DLL folders from pip's `nvidia-*` packages at start-up (top of `transcribe.py`)
A small local judge gave everything 7–8/10	Test any judge on a clearly good and a clearly bad example first; use a stronger model where scores do not spread
PC ran out of RAM (16 GB) with Whisper + Chrome + Claude	One heavy job at a time; jobs wait for free RAM; the loop may close Chrome politely (tabs restore with Ctrl+Shift+T)

2.8 What you can say about speed

Measured: Claude needs about 18,000 tokens to understand an 18:50 film from our outputs, instead of about 255,000 (one frame per shot) or 791,000 (one frame per second). Our estimate: a student who reads the outputs grasps a 15-minute video in about 300–400 seconds. If anyone asks for proof, time three students on one video with and without the outputs: that is a fair, simple test.



Tokens Claude must read to understand an 18:50 documentary (256 shots). Images ≈ width×height/750 tokens; text ≈ bytes/4.

Figure 6. The model reads 2–7% of what a frame-by-frame approach would cost, with no information thrown away: the numbers cover every frame.

Part 3. Interview questions you may get

Why not just give the video to an AI?

Most language models cannot take video, and sending every frame is very expensive. We measure everything locally for free and send a one-page summary, which is 2–7% of the cost.

What is the "ontology" exactly?

A map of notes and their relations: each note names its parent, and links name the cross relations. One vault is generated from a Python file that defines domains and concepts.

How does the system "relearn"?

Every mistake becomes a lesson row; a repeated lesson becomes a script, a checklist rule or a preference, and the checklist runs before anything is called done.

How do several accounts share context?

The notes and the global rules are private Git repos. Each PC pulls before work and pushes after, so any account on any PC starts from the same notes.

Is the downloading legal and safe?

We use our own logged-in session from a home connection, go slowly, back off on warnings and stop on any block. We store comments without usernames and delete videos after analysis. Platform terms still apply, and we only use this for research and study.

What would you improve?

Validate the camera-motion thresholds against hand labels, run a proper timed study for the 300–400 s figure, and make the paths configurable so the code runs on any machine.

What did you personally learn?

Measure before you trust a model, keep context in files instead of chats, and make the cheap tools do the volume so the expensive model only does the thinking.

Part 4. Glossary

Word	Meaning
Token	The unit AI models read and charge for; about 4 characters of English. An image costs about width×height/750 tokens.
Ontology	A structured map of concepts and how they relate.
Harness	The notes, scripts and checks around an AI model that make it work reliably over time.
Whisper	OpenAI's open speech-recognition model; faster-whisper is a faster version of it.
Demucs	Meta's open model that separates voice from music.
Ollama	A free program that runs AI models such as Qwen on your own PC.
Qwen3-VL	Alibaba's open vision-language model: it looks at an image and answers in text.
Optical flow	How each pixel moves between two frames; it reveals camera moves.
Contact sheet	Many small frames on one image, like a photographer's proof sheet.
VRAM	The graphics card's own memory; the limit on which models can run at once.
Cookies	Small files a site stores to remember that you are logged in; treat them like a password.